



NATIONAL UNIVERSITY OF TECHNOLOGY

GROUP MEMBERS:

1. Sawaira Anwar (F24605041)
2. Javeria Faisal (F24605017)
3. Bisma Fatima (F24605036)
4. Ayesha Imran (F24605019)

CLASS:

CS-24(A)

COURSE:

Software Engineering

Course Code:

CS250

SUBMITTED TO:

Ma'am SABA FAROOQ ABBASI

DATE:

December-22,2025

Project Task: Real-World Problem-Solving with Software Engineering Principles

Phase 1: Problem Identification and Definition

1.1 Problem Identification

Currently, art enthusiasts and students face significant barriers when trying to explore a diverse range of high-quality digital and traditional artwork. Most existing platforms require mandatory account creation, include intrusive advertisements, or lack specialized categories like traditional stone and marble crafts. This makes it difficult for users to find authentic, high-resolution inspiration in a single, well-organized environment. A software solution is needed to provide an open, immersive, and categorized gallery experience that prioritizes user accessibility and visual quality.

1.2 Detailed Project Description

I. Problem Statement

- **Affected Parties:** The primary groups affected are art students, digital designers, and casual art lovers.
- **Impact:** These users suffer from inefficient search processes and a lack of high-fidelity resources for traditional crafts, which hinders their creative and academic research.

II. Proposed Solution

The proposed software, ArtNet, is a web-based interactive gallery designed to streamline art discovery. The system utilizes a modern frontend stack to deliver features such as:

- A dynamic image gallery with real-time keyword searching.
- An interactive Lightbox preview system for full-screen exploration.
- High-resolution download capabilities for offline use.
- Curated highlights of traditional architecture and marble crafts.

III. Scope

The initial release of ArtNet is defined by the following boundaries:

- **Included (In-Scope):** User registration, interactive gallery search, Lightbox previewer, artist spotlight profiles, and high-res image downloading.

- **Excluded (Out-of-Scope):** User-submitted artwork uploads, e-commerce/payment gateways, and real-time social commenting features.
- **Limitations:** The initial version relies on external API sourcing for content and does not support a personal user "favorites" library.

IV. Target Audience/Users

- **Art Enthusiasts:** Casual users looking for high-quality visual inspiration.
- **Art Students:** Academic users researching traditional stone craft and architecture.
- **Registered Members:** Users who wish to utilize the full download capabilities of the site.

Phase 2: Requirements Elicitation and Specification

2.1 Functional Requirements (FRs)

These requirements describe the specific behaviors and features your software must perform:

- **FR-01 (User Registration):** The system shall allow new users to create an account by providing their first name, last name, birthday, gender, email, and phone number.
- **FR-02 (Keyword Search):** The system shall provide a search bar that allows users to filter the image gallery based on specific text keywords.
- **FR-03 (Image Preview):** The system shall provide a "Lightbox" feature to display a larger, high-resolution preview of an image when a gallery card is clicked.
- **FR-04 (Content Categorization):** The system shall organize artwork into distinct sections such as "Top Artists," "Gallery Highlights," and specific craft categories like Stone and Marble.
- **FR-05 (Image Download):** The system shall provide a download button on each image card to allow users to save high-resolution assets to their local storage.
- **FR-06 (Navigation Control):** The system shall allow users to navigate between the Home, Gallery, Highlights, and Artists pages using a persistent navigation bar or a hamburger menu on smaller screens.
- **FR-07 (Data Persistence):** The system shall capture user input from the registration form and store it as a structured data object before redirecting to the main interface.

2.2 Non-Functional Requirements (NFRs)

These requirements describe the constraints and quality attributes of your system:

- **NFR-01 (Usability):** The system shall provide a responsive user interface that adapts its layout (e.g., using a hamburger menu) for mobile and tablet devices with screens smaller than 1140px.
- **NFR-02 (Performance):** The system shall display a loading spinner/screen to the user while fetching and rendering high-resolution art assets to manage perceived wait times.
- **NFR-03 (Maintainability):** The system shall follow a modular code structure, separating logic into dedicated files for styles (.css), scripts (.js), and content (.html) for easier future updates.
- **NFR-04 (Security):** The system shall enforce data entry validation by using the required attribute on all critical input fields within the registration form.
- **NFR-05 (Visual Design):** The system shall utilize modern design techniques, such as backdrop filters (blur(5px)) and keyframe animations, to ensure an immersive and professional user experience.

Phase 3: Analysis and Design Modeling

3.1 System Design Overview

The ArtNet system is designed using a modular frontend architecture. The high-level design focuses on an immersive user experience, utilizing CSS3 Grid and Flexbox for layout management and JavaScript for dynamic state changes such as search filtering and Lightbox toggling.

3.2 System Architecture Diagram

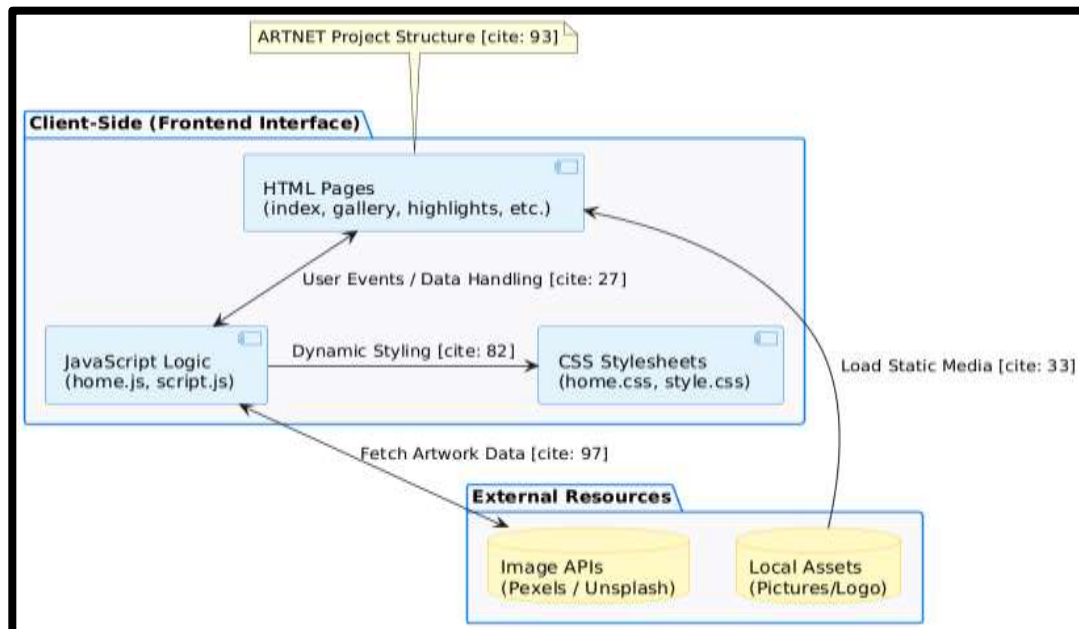
Code of System Architecture Diagram

```

1  @startuml
2  skinparam componentStyle uml2
3  skinparam package {
4      BackgroundColor #F8F9FA
5      BorderColor #007BFF
6  }
7  skinparam component {
8      BackgroundColor #E3F2FD
9      BorderColor #1976D2
10 }
11 skinparam database {
12     BackgroundColor #FFF9C4
13     BorderColor #FBC02D
14 }
15
16 package "Client-Side (Frontend Interface)" {
17     [HTML Pages\n(index, gallery, highlights, etc.)] as html
18     [CSS Stylesheets\n(home.css, style.css)] as css
19     [JavaScript Logic\n(home.js, script.js)] as js
20 }
21
22 package "External Resources" {
23     database "Image APIs\n(Pexels / Unsplash)" as api
24     database "Local Assets\n(Pictures/Logo)" as assets
25 }
26
27 html <--> js : User Events / Data Handling [cite: 27]
28 js --right--> css : Dynamic Styling [cite: 82]
29 js <--> api : Fetch Artwork Data [cite: 97]
30 html <-- assets : Load Static Media [cite: 33]
31
32 note top of html : ARTNET Project Structure [cite: 93]
33 @enduml

```

System Architecture Diagram



3.3 Database Design

While ArtNet is primarily a frontend-driven application, it follows a structured data model to ensure consistency. The system manages two primary data entities through JavaScript objects and API responses.

3.3.1 Entity Relationship Diagram (ERD)

The following diagram illustrates the logical relationship between the User and the Art Gallery assets.

3.3.2 Data Dictionary (Table Descriptions)

1. User Entity (Captured via Signup Form) This table describes the data collected during the user registration process.

Field Name	Data Type	Constraints	Description
firstName	String	Required	User's first name
lastName	String	Required	User's last name
email	String	Required, Unique	User's primary contact and identifier
birthday	Date	Required	Date of birth for profile verification
gender	String	Required	Selection of Male or Female
phone	String	Required	Contact number for the account

2. Image Metadata Entity (Managed via Gallery logic) This entity structures the information for every piece of art displayed in the gallery.

Field Name	Data Type	Description
imageID	Integer	Unique identifier for the specific artwork
sourceURL	String	The direct link to the image file used for display and download
photographer	String	The name of the artist or photographer to be credited
category	String	The tag used for filtering (e.g., "Stone Craft", "Art")

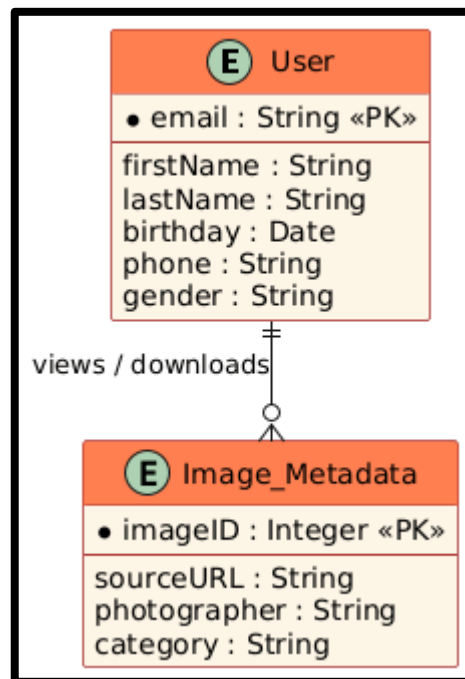
Erd code

```

1  @startuml
2  skinparam linetype ortho
3  skinparam class {
4      BackgroundColor #FDF5E6
5      BorderColor #A52A2A
6      HeaderBackgroundColor #FF7F50
7  }
8
9  entity "User" as user {
10     * email : String <<PK>>
11     --
12     firstName : String
13     lastName : String
14     birthday : Date
15     phone : String
16     gender : String
17 }
18
19 entity "Image_Metadata" as img {
20     * imageID : Integer <<PK>>
21     --
22     sourceURL : String
23     photographer : String
24     category : String
25 }
26
27 user ||--o{ img : "views / downloads"
28 @enduml

```

Erd Diagram



3.4 UML Diagrams

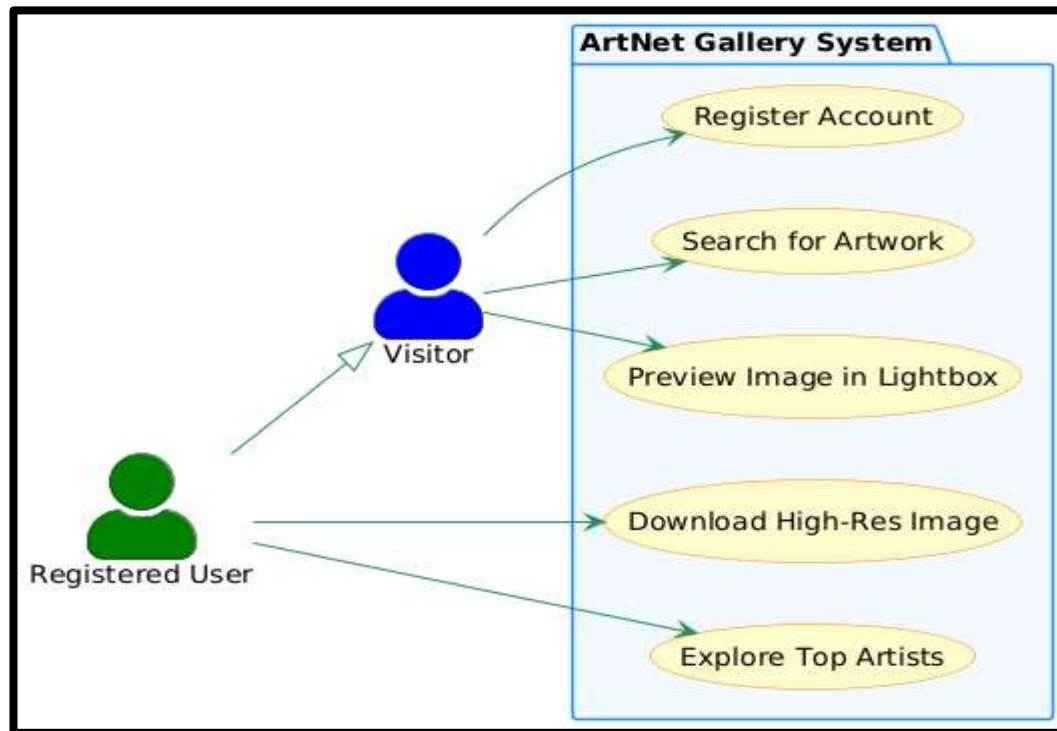
3.4.1 Use Case Diagram

Code of Use Case Diagram

```

1  @startuml
2  skinparam actorStyle awesome
3  skinparam package {
4      BackgroundColor #F0F8FF
5      BorderColor #1E90FF
6  }
7  skinparam usecase {
8      BackgroundColor #FFFACD
9      BorderColor #FF8C00
10     ArrowColor #2E8B57
11 }
12
13 left to right direction
14
15 actor "Visitor" as V #Blue
16 actor "Registered User" as U #Green
17
18 package "ArtNet Gallery System" {
19     usecase "Search for Artwork" as UC1
20     usecase "Register Account" as UC2
21     usecase "Preview Image in Lightbox" as UC3
22     usecase "Download High-Res Image" as UC4
23     usecase "Explore Top Artists" as UC5
24 }
25
26 V --> UC1
27 V --> UC2
28 V --> UC3
29 U --|> V
30 U --> UC4
31 U --> UC5
32 @enduml
  
```


Use case Diagram



Explanation:

Actors: Visitor and Registered User.

System Boundary: The ArtNet Web Application.

3.4.2 Use Case Descriptions

Use Case: Register Account.

Primary Actor: Visitor.

Flow of Events:

Visitor navigates to the Registration page.

Visitor enters required personal details.

System validates input and captures data.

System redirects the user to the Home page.

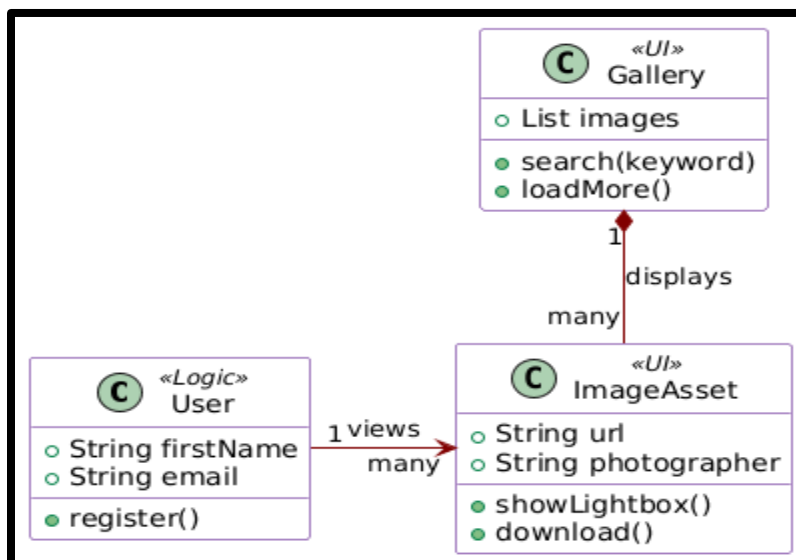
3.4.3 Class Diagram

Code of Class Diagram:

```
1 @startuml
2 skinparam class {
3     BackgroundColor White<<Logic>>
4     BackgroundColor #E6E6FA<<UI>>
5     BorderColor #4B0082
6     ArrowColor #800000
7 }
8
9 class User <<Logic>> {
10     +String firstName
11     +String email
12     +register()
13 }
14
15 class Gallery <<UI>> {
16     +List images
17     +search(keyword)
18     +loadMore()
19 }
20
21 class ImageAsset <<UI>> {
22     +String url
23     +String photographer
24     +showLightbox()
25     +download()
26 }
27
28 User "1" --> "many" ImageAsset : views
29 Gallery "1" *-- "many" ImageAsset : displays
30 @enduml
```

Classes: User (handles signup logic), Gallery (handles search logic), and ImageAsset (handles download and preview logic).

Diagram of Class



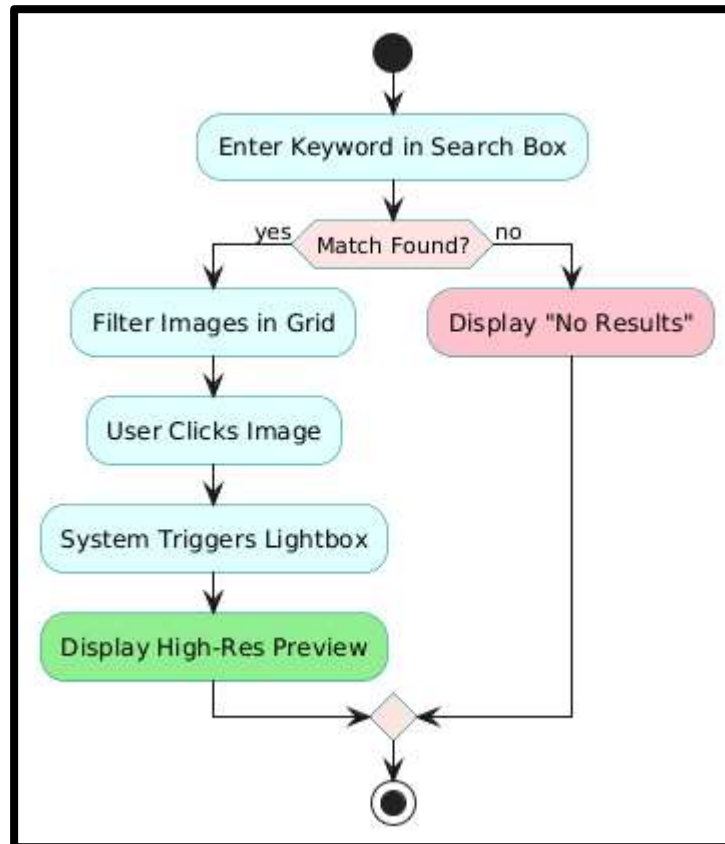
3.4.4 Activity Diagrams

Code of Activity Diagram:

```
1 @startuml
2   skinparam activity {
3     BackgroundColor #E0FFFF
4     BorderColor #008B8B
5     DiamondBackgroundColor #FFE4E1
6   }
7
8   start
9   :Enter Keyword in Search Box;
10  if (Match Found?) then (yes)
11    :Filter Images in Grid;
12    :User Clicks Image;
13    :System Triggers Lightbox;
14    #LightGreen:Display High-Res Preview;
15  else (no)
16    #Pink:Display "No Results";
17  endif
18  stop
19 @enduml
```

Workflow: Describes the sequence from entering a search keyword to the system updating the gallery grid and the user opening a Lightbox preview.

Diagram of Activity:

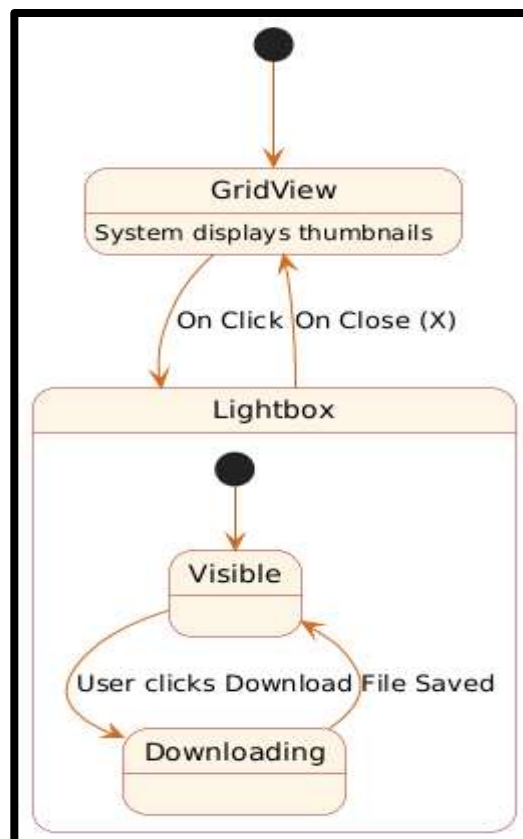


3.4.5 State Machine Diagram

Code of state Machine:

```
1 @startuml
2 skinparam state {
3     BackgroundColor #FDF5E6
4     BorderColor #A52A2A
5     ArrowColor #D2691E
6 }
7
8 [*] --> GridView
9 GridView : System displays thumbnails
10
11 state Lightbox <<preview>> {
12     [*] --> Visible
13     Visible --> Downloading : User clicks Download
14     Downloading --> Visible : File Saved
15 }
16
17 GridView --> Lightbox : On Click
18 Lightbox --> GridView : On Close (X)
19 @enduml
```

Diagram of state Machine:



Phase 4: Implementation, Testing, and Deployment

5. Implementation

5.1 Technology Stack

- **Languages:** HTML5 (Structure), CSS3 (Styling/Animations), and JavaScript (Logic).
- **Libraries:** jQuery for DOM manipulation and GSAP for high-end scrolling animations in the highlights section.
- **APIs:** Pexels/Unsplash API integration for dynamic image fetching.
- **Tools:** VS Code, Chrome DevTools, and GitHub for version control.

5.2 System Architecture / Folder Structure

Based on your project files, the project is organized as follows:

- **Root Directory:** Contains all HTML files (index.html, gallery.html, form.html, etc.).
- **Style Assets:** CSS files for specific page behaviors (home.css, style.css, hero_section_3.css).
- **Logic Assets:** Script files managing the gallery and hero sections (script.js, home.js, hero_section_3.js).

6. Testing

6.1 Test Plan

The testing strategy focuses on **Functional Testing** to ensure links and forms work, and **UI/UX Testing** to verify responsiveness and animations across different screen sizes.

6.2 Test Cases (Minimum 10 Required)

Test Case ID	Test Description	Preconditions	Test Steps	Expected Result	Actual Result
TC-01	User Registration Submission	Signup page is open ⁷ .	1. Fill all fields. 2. Click 'Register'.	Redirects to home page.	Pass

TC-02	Empty Field Validation	Signup page is open ⁸ .	1. Leave Email empty. 2. Click 'Register'.	Browser shows "Required" prompt.	Pass
TC-03	Image Search Filter	Gallery page is open ⁹ .	1. Type "Nature" in search box.	Only matching images remain visible.	Pass
TC-04	Lightbox Activation	Gallery grid is loaded ¹⁰ .	1. Click on any image card.	Lightbox modal opens with full image.	Pass
TC-05	Image Download	Lightbox is active ¹¹ .	1. Click the 'Import' icon.	Browser starts file download.	Pass
TC-06	Responsive Navigation	Viewport < 1140px ¹² .	1. Open home page on mobile.	Nav-links hide; Hamburger icon appears.	Pass
TC-07	Hamburger Menu Toggle	Mobile view active ¹³ .	1. Click Hamburger icon.	Sidebar menu slides into view.	Pass
TC-08	Page Scroll Animation	Home page open ¹⁴ .	1. Scroll to 'About Us'.	Text fades in and moves up.	Pass
TC-09	Artist Page Navigation	Artist page open ¹⁵ .	1. Click 'Explore More' button.	User is navigated to Gallery page.	Pass
TC-10	External API Loading	Internet connected ¹⁶ .	1. Open Gallery page.	API images load into grid successfully.	Pass

6.3 Testing Tools Used

- **Manual Testing:** Conducted via Chrome DevTools to verify responsive breakpoints and element behaviors.
- **Browser Console:** Used to debug JavaScript event listeners and API responses.

7. Deployment

7.1 Deployment Method

The project is deployed using **GitHub Pages** (or Netlify/Render), which provides a live link for the instructor to view the functional interface.

7.2 GitHub Repository Link

- **GitHub Repository URL:** [GitHub Pages](#)
- **Collaborators Added:**
 1. **Instructor:** Saba Farooq (GitHub: saimayasmeeen4)
 2. **Teammate 1** Sawaira Anwar (F24605041)
 3. **Teammate 2** Javeria Faisal (F24605017)
 4. **Teammate 3** Bisma Fatima (F24605036)
 5. **Teammate 4** Ayesha Imran (F24605019)

8. Future Enhancements

List possible improvements or new features that can be added later:

- **User Upload Dashboard:** Develop a secure portal where registered artists can upload their own artwork directly to the platform rather than relying on external APIs.
- **E-commerce Integration:** Add a payment gateway (e.g., Stripe) to allow users to purchase physical prints or digital licenses of the showcased art.
- **Social Interactions:** Implement a "Like" and "Comment" system to allow users to interact with artists and build a community.
- **AI Search:** Integrate machine learning to allow users to search for artwork by uploading an image rather than using keywords.
- **Personal Gallery:** Allow registered users to create "Favorites" collections saved to their profiles.

9. Conclusion

Summarize the problem solved, main features built, technologies used, and key learning outcomes:

- **Problem Solved:** ArtNet successfully provides a centralized, high-quality, and accessible digital art repository that removes registration barriers for casual browsers.
- **Main Features:** The system delivers a responsive interface, keyword-based art searching, an interactive Lightbox previewer, and a functional user registration system.
- **Technologies Used:** The project was implemented using a modern frontend stack consisting of HTML5, CSS3 (using backdrop filters and animations), and JavaScript with GSAP and jQuery libraries.
- **Learning Outcomes:** The team successfully applied the full Software Development Life Cycle (SDLC), moving from requirements elicitation to UML modeling and final deployment on GitHub.

10. References

List all information sources used for this project:

- **MDN Web Docs:** "HTML5 and CSS3 Documentation," [Online]. Available: <https://developer.mozilla.org>.
- **GSAP Documentation:** "GreenSock Animation Platform Reference Guide for UI Animations," [Online]. Available: <https://greensock.com/docs>.
- **Pexels API:** "Image Sourcing and Documentation," [Online]. Available: <https://www.pexels.com/api>.
- **Course Materials:** Software Engineering Class Notes, Department of Computer Science, Nutech, 2025.